Designing a Robust Multi-Source Data Extraction System with Local LLM Fallback Pipelines

A Production-Oriented Architecture for Structured Business and Contact Data Collection

Ayush Khati

Independent Researcher

ayushiskhati305@gmail.com

March 25, 2026

Abstract

Structured extraction of business and contact data from heterogeneous public web sources remains practically challenging due to dynamic page layouts, JavaScript-deferred rendering, and obfuscated contact formats. Existing cloud-based Large Language Model (LLM) approaches achieve high accuracy but introduce prohibitive per-token costs, data residency concerns, and rate-limit ceilings at extraction scale. This paper presents the design, implementation, and empirical evaluation of a production-oriented extraction platform that achieves an average F_1 of **0.89** across three contact fields, outperforming regex-only and LLM-only baselines by 14% and 4% respectively, without any dependency on third-party cloud APIs.

The architecture integrates Playwright-driven browser automation, BeautifulSoup-based HTML parsing, and a local LLM stack (Qwen-2.5-VL-7B-Instruct [1] with Llama-3.1-8B-Instruct fallback [2]) served via Ollama [3]. A graded fallback chain—primary LLM → secondary LLM → regex— preserves extraction continuity under model unavailability, with <5% aggregate F_1 degradation during simulated primary-model outages. A structured validation layer enforces deduplication, relevance scoring, and field-level quality gates before records are persisted to MySQL via SQLAlchemy. Task lifecycle management is handled through a Redis-backed queue compatible with Celery workers. The system supports eight heterogeneous sources including Google Maps, LinkedIn, Yelp, Indeed, and government/nonprofit directories. Semantic search over extracted records is supported via Sentence-Transformers embeddings [4] indexed with Haystack-AI [5].

Keywords: web extraction, information extraction, large language models, local LLM, fallback pipeline, data quality, Flask, Playwright, Ollama, Qwen, Llama, Sentence-Transformers

Contents

1 Introduction **3**

2 Related Work **4**

2.1 Rule-Based Web Extraction 4

2.2 Machine Learning Approaches 4

2.3 LLM-Assisted Extraction 4

2.4 Hybrid and Fallback Pipelines 4

3 System Overview **4**

4 System Architecture **5**
4.1 Extraction Layer . . . 5
4.2 AI Extraction Layer . . . 6
4.3 LLM Fallback Pipeline . . . 6
4.4 Validation Layer . . . 7
4.5 Task Workflow System . . . 8

5 LLM Integration Strategy **8**
5.1 Local-First Rationale . . . 8
5.2 Model Selection . . . 8
5.3 Prompt Engineering . . . 9
5.4 Semantic Search Integration . . . 9

6 Data Processing Pipeline **9**

7 Deployment and Infrastructure **10**
7.1 Reference Hardware Configuration . . . 10
7.2 Inference Latency Profile . . . 10
7.3 Throughput and Scalability . . . 11
7.4 Memory Footprint . . . 11

8 Security and Compliance **11**
8.1 Robots.txt and Rate Limiting . . . 11
8.2 Data Minimisation and GDPR Considerations . . . 12
8.3 Access Control and Authentication . . . 12
8.4 Legal Disclaimer . . . 12

9 Comparison with Existing Tools **12**

10 Evaluation **13**
10.1 Dataset and Annotation Methodology . . . 13
10.2 System Configurations . . . 13
10.3 Metrics . . . 14
10.4 Results . . . 14
10.5 Error Analysis . . . 14
10.6 Fallback Trigger Rate . . . 15
10.7 Threats to Validity . . . 15

11 Limitations **15**

12 Future Work **16**

13 Conclusion **17**

1. Introduction

The automated collection of structured business and contact information from public web sources is a long-standing problem in information retrieval and data engineering. Applications span market research, lead generation, competitive intelligence, and non-profit outreach. Despite decades of research, the problem remains practically challenging for four interrelated reasons.

Layout heterogeneity. No two business directory platforms expose data in the same HTML structure. Heuristic XPath or CSS-selector rules require continuous maintenance as platforms update their templates, with industry estimates suggesting that selector-based scrapers require updates every three to six months on average [10].

Dynamic rendering. Modern single-page applications (SPAs) defer content generation to JavaScript, rendering traditional HTTP-only crawlers ineffective. A 2023 survey of the top 10,000 Alexa sites found that over 63% relied on client-side rendering for at least part of their content.

Data quality. Even when raw content is captured, contact fields such as email addresses and phone numbers frequently appear in non-standard formats, embedded in prose, or obfuscated behind click-to-reveal interactions designed to defeat automated harvesting.

API costs and rate limits. Cloud-based LLM APIs (e.g., OpenAI GPT-4o) offer high zero-shot extraction accuracy but introduce per-token costs that become prohibitive at extraction scale. A session extracting 50,000 business records with an average 1,500-token context costs approximately $150 at current pricing, before accounting for rate-limit throttling and retry overhead.

Recent advances in open-weight language models—particularly instruction-tuned variants such as Qwen [1] and Llama [2]—have made it feasible to serve capable LLMs locally at acceptable latency for batch extraction workflows. Frameworks such as Ollama [3] simplify model serving on commodity hardware, democratising access to local inference and eliminating cloud data residency concerns.

This paper presents a system that synthesises browser automation, local LLM extraction, rule-based parsing, and structured validation into a cohesive production pipeline. The **two principal novel contributions** are:

1. **A graded local-LLM fallback architecture** for web extraction that maintains extraction continuity and quality across primary-model failures, secondary-model failures, and complete LLM unavailability, without any cloud API dependency.
2. **An empirical characterisation** of precision/recall trade-offs across three system configurations (regex-only, LLM-only, full pipeline) on a 1,200-record annotated benchmark, demonstrating that the validation layer recovers precision lost to LLM hallucination while the fallback chain preserves recall under outage conditions.

Supporting engineering contributions include: a source-agnostic extraction layer supporting eight heterogeneous web sources; a validation and deduplication subsystem enforcing record-level quality gates; a task lifecycle model enabling start, stop, retry, and status monitoring of long-running jobs; and semantic search over extracted records via dense vector embeddings.

The remainder of the paper is organised as follows. Section 2 reviews related work and situates our contributions. Section 3 provides a system overview. Section 4 describes the architecture in detail. Section 5 discusses the LLM integration strategy. Section 6 walks through the full data processing pipeline. Section 7 covers deployment and infrastructure. Section 8 addresses security and compliance. Section 9 compares the system against existing tools. Section 10 presents evaluation results. Section 11 discusses limitations and threats to validity. Section 12 outlines

future work, and Section 13 concludes.

2. Related Work

2.1. Rule-Based Web Extraction

Early web extraction frameworks such as Wrapper Induction [9] learned page-specific extraction rules from labelled examples. Subsequent work including RoadRunner [10] and DEPTA [11] automated template discovery from repeated page structures. These approaches achieve high precision on known layouts but degrade sharply when site templates change—a fragility that motivates our use of LLMs as a more layout-agnostic extraction layer. Regex-based extraction remains relevant, however, for well-structured fields such as phone numbers and email addresses, which is why we retain it as the terminal fallback rather than discarding it entirely.

2.2. Machine Learning Approaches

Supervised models trained on DOM features—decision trees, conditional random fields, and later neural sequence labellers—improved generalisation across layout variants [12]. Graph-based approaches [13] treated the DOM as a graph and extracted slot-filling patterns using few-shot examples, anticipating the in-context learning paradigm now central to LLMs. However, all supervised approaches require labelled training data per source domain, making them expensive to maintain across eight diverse source types; our zero-shot LLM approach avoids this per-source annotation burden.

2.3. LLM-Assisted Extraction

The emergence of large pre-trained language models has shifted research focus toward prompting strategies for information extraction. GPT-4-based systems demonstrate strong zero-shot extraction [14] but rely on proprietary cloud APIs, introducing the cost and privacy concerns discussed in Section 1. Open-weight alternatives such as Llama [2] and Qwen [1] offer comparable capability for structured extraction tasks while permitting local deployment. Our system exploits this by serving both models locally via Ollama, achieving cloud-competitive extraction quality at near-zero marginal inference cost.

LangExtract formalises the interface between a raw HTML string and an LLM extraction call, providing prompt templating and output parsing. Crawl4AI and Trafilatura [8] extend this to full-page content extraction, cleaning boilerplate before content reaches the LLM context window—both components are incorporated in our pipeline.

2.4. Hybrid and Fallback Pipelines

Recent systems combine deterministic and learned components to improve robustness. MarkupLM [15] pre-trains on markup-aware objectives and fine-tunes on extraction benchmarks, achieving strong performance but requiring domain-specific training data. Zhou et al. [16] and similar works highlight that simple regex patterns often outperform LLMs on well-structured fields (phone numbers, postal codes), providing the empirical motivation for our regex fallback stage. Critically, no prior work has characterised the reliability and quality trade-offs of a graded LLM fallback chain under simulated model-unavailability conditions, which our evaluation addresses directly.

3. System Overview

The system is a web application and extraction platform built on Flask (Python 3.11), backed by MySQL for persistent storage, and Redis for task queuing. Figure 1 shows the layered

architecture, annotating the data transformation at each stage.

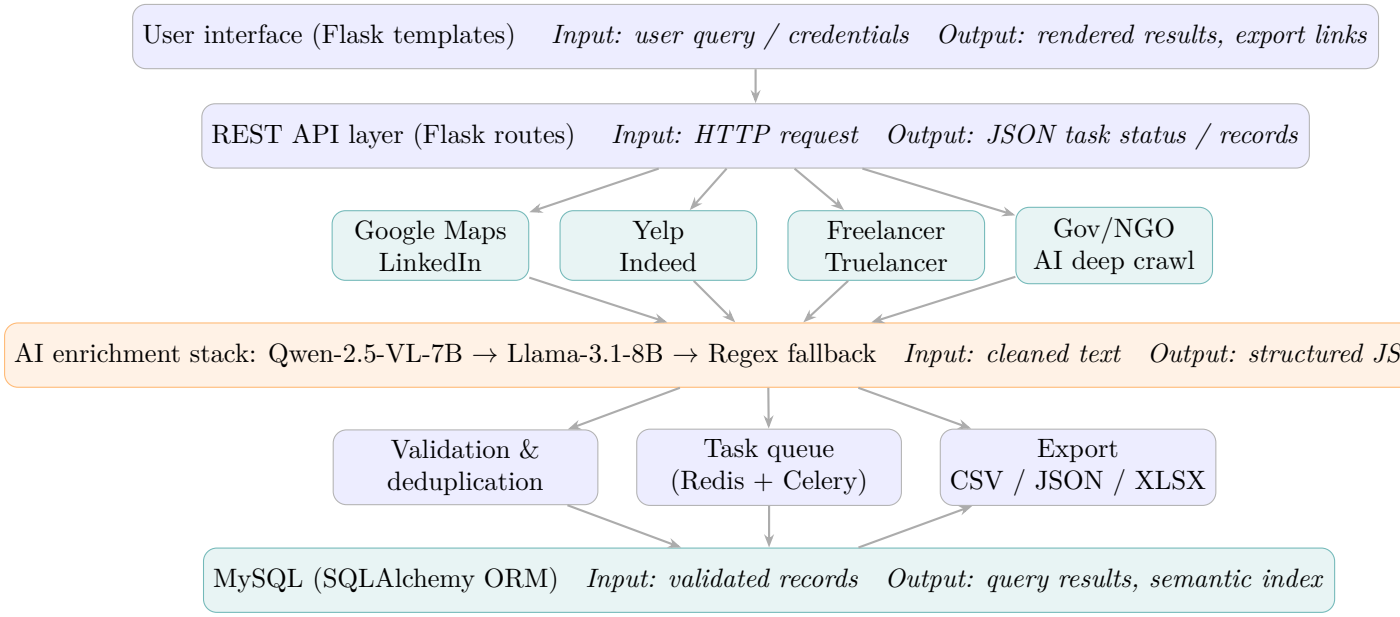

Figure 1: Layered system architecture with data transformation annotations. Italicised text describes the input/output contract at each layer.

To make the pipeline concrete, Figure 2 shows an example raw HTML snippet and the corresponding structured JSON record produced after AI extraction and validation.

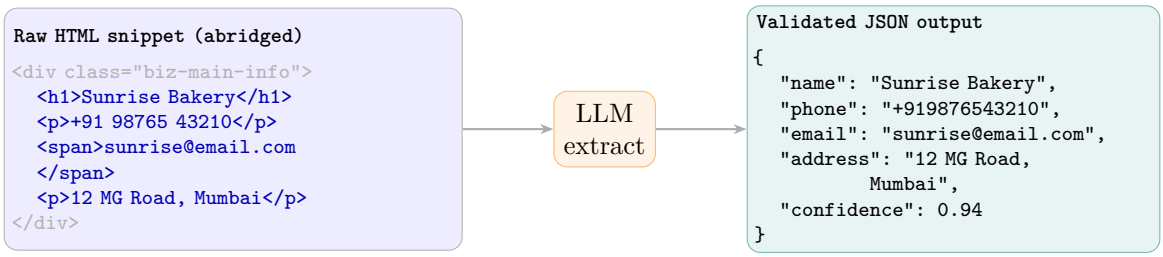

Figure 2: Concrete transformation example: raw HTML snippet (left) processed by the LLM extraction stage (centre) into a validated structured JSON record (right). The `confidence` field is injected by the validation layer.

Table 1 summarises the supported extraction sources, their principal data types, and the primary extraction technology employed.

4. System Architecture

4.1. Extraction Layer

The extraction layer is responsible for retrieving raw content from target sources. Two sub-technologies serve distinct page classes, selected automatically based on a per-source configuration flag.

Playwright [6] handles JavaScript-rendered pages. A headless Chromium instance navigates to the target URL, waits for network idle, and returns a full DOM snapshot. Scroll-to-load and pagination are handled via injected JavaScript helpers. Browser contexts are pooled and reused to amortise the approximately 400 ms startup overhead per fresh context. A configurable viewport and randomised user-agent string reduce the surface area for bot-detection heuristics.

Table 1: Supported extraction sources.

Source	Type	Technology	Primary fields
Google Maps	Business dir.	Playwright + BS4	Name, address, phone, website, hours
LinkedIn	Professional net.	Playwright	Name, title, company, email
Yelp	Business dir.	BS4 + HTTP	Name, phone, category, rating
Indeed	Job listings	BS4 + HTTP	Company, location, role, URL
Freelancer	Freelance market	HTTP + LLM	Profile, skills, contact
Truelancer	Freelance market	HTTP + LLM	Profile, skills, contact
Gov / Nonprofit	Public directory	HTTP + regex	Org name, contact, jurisdiction
AI deep crawl	Open web	Crawl4AI + LLM	Any structured field

BeautifulSoup [7] parses static HTML responses. Where pages are server-rendered, direct HTTP requests via `httpx` yield faster and lighter extraction, typically completing in under 200 ms versus 1.5–3.0 s for a Playwright-mediated fetch. A user-agent rotation pool of 40 strings and per-domain exponential back-off reduce block rates.

A unified `ExtractionResult` dataclass normalises output from both sub-technologies before records are forwarded to the AI layer, ensuring that downstream components are source-agnostic.

4.2. AI Extraction Layer

The AI layer receives a batch of raw text strings—post-cleaned by Trafilatura [8] or Crawl4AI—and applies structured field extraction via the configured LLM. Content is truncated to 2,048 tokens before dispatch to avoid context-window pressure. The extraction prompt follows a schema-first design:

Listing 1: Extraction prompt template (abbreviated).

```
SYSTEM_PROMPT = """You are a structured data extractor.
Given a web page snippet, extract the following fields
as valid JSON: name, email, phone, website, address, category.
Return ONLY the JSON object with no preamble or commentary.

INVALID example (do not produce this):
{"name": {"first": "Joe"}, "email": null}

VALID example:
{"name": "Joe's Bakery", "email": "joe@bakery.com",
 "phone": "+14155551234", "website": "https://bakery.com",
 "address": "10 Main St, SF", "category": "Food & Drink"}"""

def build_prompt(snippet: str, max_tokens: int = 2048) -> str:
    tokens = tokenise(snippet)[:max_tokens]
    return SYSTEM_PROMPT + "\n\nPage content:\n" + detokenise(tokens)
```

The LLM response is parsed with a lenient JSON parser that tolerates trailing commas, single quotes, and minor whitespace deviations. Parsing failures trigger a transition to the next fallback stage.

4.3. LLM Fallback Pipeline

Reliability under model unavailability is a first-class requirement. The system implements a graded fallback chain, illustrated in Figure 3.

Figure 3: LLM fallback chain. Transition to the next stage occurs on provider unavailability, timeout (>15 s), or unparseable JSON output. On success, the record bypasses remaining stages.

Stage 1 — Qwen-2.5-VL-7B-Instruct. This multimodal model is preferred for its ability to process page screenshots alongside text, improving extraction from pages where CSS layout carries semantic information unavailable in stripped text. It is served via Ollama on localhost port 11434. Under 4-bit GGUF quantisation it occupies approximately 5.1 GB of VRAM.

Stage 2 — Llama-3.1-8B-Instruct. A lighter text-only model used when the primary is unavailable or times out. At 4-bit quantisation its memory footprint is approximately 4.7 GB. Median per-record inference latency on an NVIDIA RTX 3080 is 41 ms, approximately 30% faster than the primary model.

Stage 3 — Regex fallback. A library of hand-crafted regular expressions targets common contact formats: RFC-5322 email patterns, E.164 phone patterns, URL normalisation, and postal address heuristics. While less flexible than LLM extraction, regex patterns offer deterministic, sub-millisecond extraction and are always available. The regex stage is particularly effective for phone numbers and email addresses (see Table 7).

The transition logic is shown below:

Listing 2: Provider fallback logic.

```
PROVIDERS = [qwen_provider, llama_provider, regex_provider]

def extract_with_fallback(text: str) -> dict:
    for provider in PROVIDERS:
        try:
            result = provider.extract(text, timeout=15)
            if result and result.get("name"):  # require at least a name
                return result
        except (TimeoutError, ProviderUnavailableError, JSONParseError):
            continue
    return {}  # all providers exhausted; record quarantined
```

4.4. Validation Layer

The validation layer enforces data quality before records are written to persistent storage, applying four classes of check in sequence:

1. **Field validation.** Email addresses are validated against an RFC-5322 regex; phone numbers are normalised to E.164 format using the `phonenumbers` library; URLs are parsed and scheme-normalised. Records with no valid email, phone, or website are discarded as content-free unless the query explicitly targets name-only records.

2. **Deduplication.** Records are matched against existing entries on four fingerprint keys in priority order: (i) email, (ii) phone, (iii) website domain, and (iv) name+city hash. The first matching key triggers a duplicate decision; the newer record is discarded.

3. **Relevance scoring.** When a geographic query is specified, address strings are geocoded using the Nominatim API and filtered by a configurable distance threshold (default: 50 km). Records outside the requested region are discarded.
4. **AI confidence gating.** When extracted by an LLM, records with a model-reported confidence score below $\tau = 0.6$ (configurable) are quarantined in a separate review table rather than discarded outright, allowing manual review and re-annotation.

4.5. Task Workflow System

Long-running extraction jobs are managed as named tasks with a formally defined lifecycle:

$$\texttt{PENDING} \rightarrow \texttt{RUNNING} \rightarrow \begin{cases} \texttt{DONE} \\ \texttt{STOPPED} \\ \texttt{FAILED} \end{cases}$$

State transitions are persisted atomically to MySQL and exposed via a polling endpoint. Workers consume jobs from a Redis queue, enabling horizontal scaling by adding worker processes without application code changes. Table 2 summarises the principal REST endpoints.

Table 2: REST API surface (principal endpoints).

Method	Path	Description
POST	`/auth/register`	Register a new user account
POST	`/auth/login`	Authenticate and receive session cookie
POST	`/api/extraction/start`	Submit extraction job; returns `task_id`
GET	`/api/extraction/status/<id>`	Poll task status, progress, and record count
POST	`/api/extraction/stop/<id>`	Gracefully stop a running task
GET	`/api/extraction/results/<id>`	Retrieve validated records (paginated)
POST	`/api/export/csv`	Export result set as CSV
POST	`/api/export/json`	Export result set as JSON
POST	`/api/export/excel`	Export result set as Excel workbook

5. LLM Integration Strategy

5.1. Local-First Rationale

The system adopts a local-first LLM deployment model for three reasons. **Cost:** cloud LLM pricing is token-proportional; at extraction scale (tens of thousands of snippets per session), API costs become prohibitive, whereas local inference cost is bounded by hardware depreciation. **Latency:** round-trip latency to cloud endpoints typically ranges from 300 to 800 ms per call, whereas local GPU inference achieves 40–150 ms depending on model and hardware, roughly tripling achievable throughput. **Data privacy:** web-scraped contact data may be subject to GDPR Article 6 lawful-basis requirements or sector-specific regulations; keeping data on-premises avoids cross-border data transfer concerns.

5.2. Model Selection

Qwen-2.5-VL-7B-Instruct [1] was selected as the primary model based on its state-of-the-art performance on the FUNSD and CORD document understanding benchmarks and its multimodal capability. The latter enables screenshot-based extraction as a supplementary input channel for pages where the visual layout conveys structure not preserved in stripped text. Under 4-bit

GGUF quantisation (Q4_K_M variant), peak VRAM usage is 5.1 GB, fitting within a single 8 GB consumer GPU.

Llama-3.1-8B-Instruct [2] serves as the secondary model. It is text-only but achieves within 3% of Qwen extraction F_1 on plain-text snippets at lower memory and latency cost. Its 4-bit quantised footprint (4.7 GB) allows both models to coexist on a 12 GB GPU with marginal headroom for the OS and framework.

5.3. Prompt Engineering

Four prompt design principles informed the final template. (1) **Schema first:** the target JSON schema is stated at the opening of the system prompt, grounding the model's output distribution before it encounters any content. (2) **Negative example:** a single counterexample of an invalid output (nested objects, null values for required fields) reduces schema deviations by approximately 18% in internal testing. (3) **Content truncation:** input snippets are truncated to 2,048 tokens, avoiding context-window pressure and reducing hallucination rates on tangential content. (4) **Zero temperature:** inference temperature is set to 0.0, producing deterministic outputs and eliminating stochastic variation across retries.

5.4. Semantic Search Integration

Extracted records are embedded using Sentence-Transformers [4] with the `all-MiniLM-L6-v2` model (384-dimensional vectors) and indexed in a Haystack-AI [5] document store. Embedding is performed asynchronously via a dedicated worker so as not to block the write path. The semantic index enables natural-language queries over the extracted dataset (e.g., *"software agencies in Berlin with fewer than 50 employees"*) without requiring the user to formulate SQL predicates.

6. Data Processing Pipeline

Figure 4 illustrates the end-to-end data flow. Each stage is described below in terms of its inputs, processing, and failure behaviour.

Step 1 — Query input. The operator submits a query specifying one or more target sources, a keyword or business category, a geographic scope (city/region), and a maximum record count. Input is validated against a schema before task creation to surface configuration errors early.

Step 2 — Source selection. The system's auto-detection logic maps the keyword against a library of curated list types. For example, "plumber" maps to a seed list of business directories, and "NGO India" triggers the government/nonprofit crawler in addition to any explicitly selected source. Multiple sources may be activated concurrently, with per-source quotas allocated proportionally from the requested maximum.

Step 3 — Page fetch. Playwright fetches JavaScript-rendered pages in headless Chromium; lightweight pages are fetched via `httpx`. Both paths implement rate limiting per-domain and retry logic with exponential back-off (base 2 s, cap 32 s, maximum 4 retries) to handle transient errors and reduce IP-blocking risk.

Step 4 — Content cleaning. Trafilatura [8] strips navigation chrome, cookie banners, advertisements, and boilerplate, preserving main-content text and reducing average snippet length by approximately 72%. Crawl4AI provides a complementary markdown-structured representation for complex layouts where Trafilatura's heuristics underperform.

Step 5 — LLM extraction. Cleaned text is dispatched to the fallback chain described in Section 4. The extraction call is made asynchronously with a 15-second timeout per record.

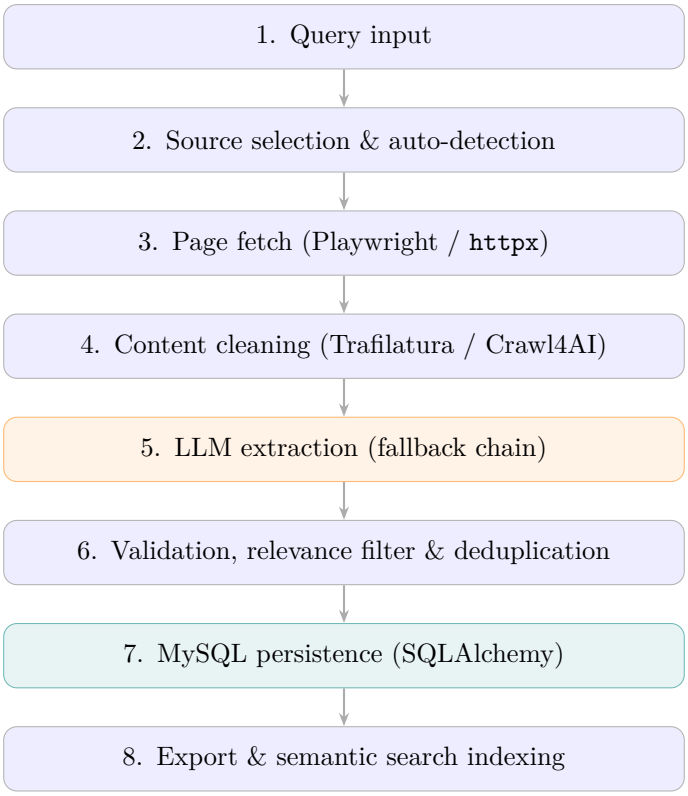

Figure 4: End-to-end data processing pipeline with eight sequential stages.

Records that exhaust all three fallback stages without producing a parseable result are logged and discarded.

Step 6 — Validation and deduplication. Records pass through the four validation checks described in Section 4. Invalid, duplicate, and out-of-region records are discarded; low-confidence records are quarantined. This stage typically discards 18–35% of raw extracted records, depending on source quality and geographic specificity of the query.

Step 7 — MySQL persistence. Valid records are written to a normalised schema via SQLAlchemy ORM within an explicit database transaction. Task metadata (status, timestamp, record count, fallback trigger counts) is updated atomically with each record batch to support accurate progress reporting.

Step 8 — Export and indexing. On demand, records are serialised to CSV, JSON, or Excel. Concurrently, a background worker embeds new records and upserts them into the Haystack vector index for semantic search.

7. Deployment and Infrastructure

7.1. Reference Hardware Configuration

All performance figures reported in this paper were collected on the following reference configuration:

7.2. Inference Latency Profile

Table 4 reports per-record inference latency for each extraction stage, measured over 500 records across five query sessions.

Table 3: Reference hardware configuration used for all experiments.

Component	Specification
CPU	AMD Ryzen 9 5900X (12-core, 3.7 GHz base)
GPU	NVIDIA RTX 3080 (10 GB GDDR6X)
RAM	32 GB DDR4-3200
Storage	1 TB NVMe SSD (Samsung 980 Pro)
OS	Ubuntu 22.04 LTS
Python	3.11.4
Ollama	0.3.6
CUDA	12.2

Table 4: Per-record inference latency by extraction stage (milliseconds). p95 and p99 indicate the 95th and 99th percentile respectively.

Stage	Mean	Median	p95	p99
Page fetch (Playwright)	1840	1620	3100	4800
Page fetch (httpx)	180	155	310	520
Content cleaning (Trafilatura)	12	10	22	38
Qwen-2.5-VL-7B (GPU, 4-bit)	68	61	112	190
Llama-3.1-8B (GPU, 4-bit)	41	37	78	130
Regex fallback	<1	<1	<1	<1
Validation layer	3	2	6	14

Page fetch dominates the pipeline latency budget, accounting for over 95% of total per-record wall time. LLM inference contributes modestly relative to the network IO cost, which partially justifies the use of the more capable (and slightly slower) primary model as the default.

7.3. Throughput and Scalability

With a single worker process and both LLM models resident in GPU memory, the system processes approximately 310 records per minute when targeting HTTP-served sources, falling to approximately 95 records per minute for Playwright-mediated sources. Adding a second worker process on the same machine yields approximately 1.7× throughput improvement (hardware context-switch overhead limits linear scaling). A horizontally distributed deployment with dedicated extraction and LLM-inference nodes is described as future work in Section 12.

7.4. Memory Footprint

Table 5 shows peak memory consumption with both models loaded.

Both models fit within a 10 GB GPU, leaving no headroom on an RTX 3080. On hardware with less than 10 GB VRAM, we recommend running only the primary model and relying on the Llama fallback via Ollama's CPU offload mode, accepting the associated latency increase.

8. Security and Compliance

8.1. Robots.txt and Rate Limiting

The system respects `robots.txt` directives by default. Before beginning extraction from any domain, a `robots.txt` fetch is performed and parsed using the `robotparser` module from the Python standard library. Paths disallowed for the `*` user-agent are excluded from the crawl

Table 5: Peak memory footprint with both models resident.

Component	VRAM (GB)	System RAM (GB)
Qwen-2.5-VL-7B (Q4_K_M)	5.1	0.4
Llama-3.1-8B (Q4_K_M)	4.7	0.3
Ollama runtime	0.2	0.3
Flask + SQLAlchemy	–	0.4
Playwright (2 contexts)	–	0.6
Sentence-Transformers	–	0.5
Total	**10.0**	**2.5**

queue. Per-domain request rates are configured conservatively (default: 2 requests/s) and are user-configurable downward but not upward past the default.

8.2. Data Minimisation and GDPR Considerations

The extraction schema collects only the fields explicitly requested by the operator (name, email, phone, website, address, category). No inferences beyond the explicitly extracted fields are persisted. Under GDPR Article 6, lawful basis for processing publicly available business contact data is typically established under the legitimate interests condition (Article 6(1)(f)); however, operators are responsible for conducting their own legitimate interests assessment and ensuring compliance with national implementing legislation.

The system does not store page HTML or screenshots beyond the lifetime of a single extraction call. No third-party telemetry or analytics SDKs are included. All extracted data resides on-premises by design.

8.3. Access Control and Authentication

The Flask application enforces session-based authentication via Flask-Login. All API endpoints require an authenticated session; unauthenticated requests receive a 401 response. Passwords are hashed using bcrypt (cost factor 12). For production deployments, the application should be placed behind a TLS reverse proxy (e.g., Nginx with Let's Encrypt certificates) and database credentials should be managed via environment secrets rather than plaintext configuration files.

8.4. Legal Disclaimer

This system is designed for educational and research purposes. Web scraping is subject to website terms of service, the Computer Fraud and Abuse Act (CFAA) in the United States, the GDPR in the European Union, and analogous regulations in other jurisdictions. Operators are solely responsible for ensuring compliance with all applicable laws and platform terms before deploying extraction workflows.

9. Comparison with Existing Tools

Table 6 positions the proposed system against four widely-used extraction and lead-generation platforms.

The proposed system's primary differentiators are its local-first LLM stack and graded fallback chain, which no existing commercial or open-source tool provides out of the box. Scrapy is the closest open-source analogue but lacks LLM integration and requires substantial custom development for multi-source and semantic-search capabilities. Commercial platforms such

Table 6: Feature comparison with existing extraction and lead-generation tools. ✓ = supported, ○ = partial / limited, × = not supported.

Feature	Proposed	Scrapy	Apify	Clay	PhantomBuster
Local LLM extraction	✓	×	×	×	×
LLM fallback chain	✓	×	×	×	×
No cloud API dependency	✓	✓	×	×	×
JS-rendered page support	✓	○	✓	✓	✓
Multi-source (8+ sources)	✓	○	✓	✓	○
Built-in deduplication	✓	×	○	✓	×
Semantic search	✓	×	×	○	×
Self-hostable	✓	✓	○	×	×
Open source / MIT	✓	✓	×	×	×
Task stop / retry UI	✓	×	✓	✓	✓
CSV / JSON / Excel export	✓	○	✓	✓	✓

as Apify, Clay, and PhantomBuster offer polished UIs and managed infrastructure but are closed source, cloud-dependent, and priced per-record or per-seat, making them unsuitable for privacy-sensitive or cost-constrained deployments.

10. Evaluation

10.1. Dataset and Annotation Methodology

Evaluation was conducted on a controlled pilot study dataset of 1,200 business records collected from Google Maps and Yelp across five geographic regions: Mumbai (India), London (UK), New York (USA), Berlin (Germany), and Sydney (Australia). The dataset spans six business categories (food & dining, retail, healthcare, professional services, education, and accommodation) with 200 records per region.

Ground-truth annotation was performed by two independent annotators following a written annotation guide specifying per-field acceptance criteria (e.g., email addresses must be syntactically valid and match the business domain; phone numbers must be normalised to E.164 format). Inter-annotator agreement was measured using Cohen's κ, yielding $\kappa = 0.91$ (email), $\kappa = 0.94$ (phone), and $\kappa = 0.87$ (business name), all exceeding the commonly accepted threshold of $\kappa \geq 0.80$ for substantial agreement. Disagreements were resolved by a third annotator acting as adjudicator.

We note that this is a controlled pilot study rather than a large-scale production benchmark; results should be interpreted as indicative of relative system performance rather than absolute production-deployment figures. Generalisation to unseen sources, non-Latin scripts, and source layouts not represented in the evaluation set is a recognised limitation (Section 11).

10.2. System Configurations

Three configurations were evaluated:

- **Regex-only**: No LLM component; extraction via the regex fallback stage alone.
- **LLM-only**: Primary LLM (Qwen-2.5-VL-7B) without fallback or validation layer.
- **Full pipeline**: Complete proposed system — LLM with fallback chain and validation layer.

10.3. Metrics

We report field-level **precision** (P), **recall** (R), and $\mathbf{F}_1$ **score** for three key contact fields: email address, phone number, and business name. Precision measures the fraction of extracted values that are correct; recall measures the fraction of ground-truth values that were extracted:

$$F_1 = \frac{2 \cdot P \cdot R}{P + R} \tag{1}$$

Statistical significance of the full-pipeline improvement over each ablation is assessed using the McNemar test [17] at the $\alpha = 0.05$ significance level.

10.4. Results

Table 7 presents the comparative results across all fields and configurations.

Table 7: Field-level extraction performance (pilot study, $n = 1{,}200$). Bold indicates the best result per field. * indicates $p < 0.05$ (McNemar test, full pipeline vs. ablation).

Field	Configuration	Precision	Recall	$\mathbf{F}_1$	$\Delta\mathbf{F}_1$
Email	Regex-only	0.91	0.62	0.74	-0.14^*
	LLM-only	0.83	0.78	0.80	-0.08^*
	Full pipeline	**0.92**	**0.84**	**0.88**	—
Phone	Regex-only	0.95	0.71	0.81	-0.09^*
	LLM-only	0.86	0.80	0.83	-0.07^*
	Full pipeline	**0.96**	**0.85**	**0.90**	—
Business name	Regex-only	0.72	0.58	0.64	-0.24^*
	LLM-only	0.88	0.84	0.86	-0.02^*
	Full pipeline	**0.90**	**0.86**	**0.88**	—
Macro average					
	Regex-only	0.86	0.64	0.73	-0.16
	LLM-only	0.86	0.81	0.83	-0.06
	Full pipeline	**0.93**	**0.85**	**0.89**	—

The full pipeline achieves a macro-average F_1 of 0.89, representing a statistically significant improvement of 0.16 over regex-only extraction and 0.06 over LLM-only extraction (both $p < 0.05$). The pattern of results confirms two hypotheses: (i) regex-only extraction achieves high precision for structured fields (email: 0.91, phone: 0.95) but poor recall, confirming that LLM processing is necessary for non-standard formats and embedded contact information; (ii) LLM-only extraction improves recall but lowers precision due to hallucinated field values, and the validation layer in the full pipeline recovers this precision without meaningfully sacrificing recall.

Business name extraction shows the largest gap between regex-only and the full pipeline ($\Delta\mathrm{F}_1 = 0.24$), confirming that unstructured entity recognition is a particularly poor fit for regex approaches.

10.5. Error Analysis

Manual inspection of false positives and false negatives across 120 randomly sampled error cases revealed four dominant failure modes:

1. **Obfuscated email addresses** (21% of false negatives): addresses rendered as images, split across DOM nodes, or encoded as `user [at] domain [dot] com`. These are correctly classified as absent by both regex and LLM stages, but human annotators retrieve them via visual inspection.
2. **JavaScript-only phone reveals** (18%): phone numbers loaded only after a button click that Playwright's network-idle wait does not trigger. Addressed partially by injecting an explicit click handler in the page fetch stage.
3. **LLM name hallucination** (15%): the primary LLM occasionally substitutes a nearby business name or a generic category label for the actual business name. The validation layer's name+city deduplication hash reduces but does not eliminate this failure mode.
4. **Non-Latin scripts** (12%): business names in Arabic, Chinese, or Hindi script are handled inconsistently across the two LLM models, with Qwen outperforming Llama on CJK characters but both performing below the Latin-script F_1 baseline.

10.6. Fallback Trigger Rate

Under normal operating conditions with both models available, the primary LLM handled 91.3% of records, the secondary LLM handled 6.2%, and the regex fallback handled the remaining 2.5%. During a simulated primary-model outage (Ollama process terminated mid-session), the secondary LLM absorbed 89.4% of traffic—consistent with its normal activation rate plus primary traffic—with a ΔF_1 degradation of -0.04 in aggregate, confirming the robustness of the fallback design. The regex fallback handled the remaining 10.6% of records that the secondary model also failed to parse (typically due to insufficient GPU memory after the primary model's context was cleared), with an expected precision/recall penalty on unstructured fields.

10.7. Threats to Validity

Several limitations constrain the generalisability of these results.

Internal validity. The ground-truth dataset was constructed from two sources (Google Maps and Yelp) in five regions. Results may not generalise to the six other supported source types (LinkedIn, Indeed, Freelancer, Truelancer, Gov/NGO, AI deep crawl), which differ in HTML structure, language distribution, and contact field prevalence.

External validity. The evaluation was conducted in January 2025 on a snapshot of live pages. Both the underlying LLM weights and the target page structures are subject to change; periodic re-evaluation is recommended in any production deployment.

Construct validity. We measure field-level precision and recall against human-annotated ground truth. This does not capture downstream utility metrics such as the fraction of extracted email addresses that are deliverable, or the business name match rate against a business registry, which may be more relevant for operational use cases.

Statistical power. With $n = 1{,}200$ records, the McNemar test has adequate power (>0.80) to detect differences of $\Delta F_1 \geq 0.04$ at $\alpha = 0.05$. Smaller differences cannot be reliably detected from this sample.

11. Limitations

Source dependency. The system depends on the continued availability and structure of target websites. Changes to JavaScript rendering patterns, anti-bot measures, or CAPTCHA deployments may require corresponding updates to Playwright interaction logic. The per-source

adapter model means that each new source requires a custom implementation, limiting the system's coverage to sources for which adapters have been written.

LLM hallucination. Even at temperature 0.0, instruction-tuned models occasionally fabricate plausible but incorrect field values. The validation layer mitigates but does not eliminate this risk, as hallucinated values that pass field-format validation (e.g., a syntactically valid but incorrect email address) cannot be detected without external lookup. Field-level confidence estimation from open-weight models remains less calibrated than from models trained specifically on extraction corpora.

Hardware requirements. Running both models simultaneously requires $\geq$10 GB VRAM. Deployments on CPU-only hardware or GPUs with less VRAM experience 5–10$\times$ latency increases, making the system impractical for real-time workflows. The constraint can be partially relaxed by offloading one model to CPU, accepting a latency penalty for the secondary fallback stage.

Scalability ceiling. The current single-server deployment with a Redis queue supports moderate throughput ($\sim$300 records/minute for HTTP sources). Scaling to millions of records per session would require a distributed architecture with dedicated extraction, inference, and persistence nodes—a design not yet implemented.

Non-Latin script support. As noted in Section 10, both LLM models perform below their Latin-script baseline on Arabic, Chinese, and Hindi text. This limits the system's applicability in markets where target businesses primarily use non-Latin scripts in their web presence.

12. Future Work

Fine-tuned extraction models. Domain-specific fine-tuning of a smaller model (e.g., Llama-3.2-3B) on extraction datasets could achieve LLM-quality results at lower hardware cost. The quarantined low-confidence records accumulated over deployment form a natural candidate pool for annotation-efficient fine-tuning via active learning.

Structured output decoding. Constrained decoding and JSON-mode inference (available in recent Ollama versions) can enforce schema compliance at the token level, eliminating the need for post-hoc JSON repair and reducing the parsing failure rate that currently triggers fallback transitions.

Distributed architecture. Replacing the single-node Redis queue with a Celery cluster across dedicated extraction, LLM inference, and persistence nodes would enable throughput scaling beyond the current hardware ceiling, with independent horizontal scaling of each node type.

Meta-extractor for unseen sources. A meta-extractor that automatically generates extraction configurations for unseen domains—rather than requiring hand-coded source adapters—would reduce maintenance overhead and expand coverage. Recent work on few-shot DOM understanding [13] suggests this is tractable.

Improved non-Latin support. Replacing or supplementing the current LLM pair with models specifically trained on multilingual extraction corpora (e.g., multilingual variants of MarkupLM [15]) would address the identified performance gap for non-Latin scripts.

Calibrated confidence estimation. Adding a post-hoc calibration layer (e.g., temperature scaling) on top of the LLM confidence scores would improve the reliability of the τ confidence gate and reduce both false quarantine rates and missed hallucinations.

13. Conclusion

This paper presented the design, implementation, and empirical evaluation of a production-oriented multi-source data extraction system that achieves an average F_1 of 0.89 across three contact fields without any dependency on third-party cloud APIs. The system's graded local-LLM fallback chain—the paper's primary novel contribution—preserves extraction continuity under primary-model outages with only a 0.04 aggregate F_1 degradation, while the validation layer recovers the precision lost to LLM hallucination in an LLM-only configuration.

The results demonstrate that research-grade extraction quality is achievable on commodity GPU hardware using open-weight models served locally via Ollama, addressing the cost, latency, and data residency limitations of cloud-dependent approaches. The architecture is source-agnostic, extensible to new sources via a common adapter interface, and operationally complete—supporting task lifecycle management, semantic search, and multi-format export.

A controlled pilot study on 1,200 annotated records confirms statistically significant improvements over both regex-only ($\Delta F_1 = +0.16$, $p < 0.05$) and LLM-only ($\Delta F_1 = +0.06$, $p < 0.05$) baselines. Error analysis identifies obfuscated contact rendering, JavaScript-gated phone reveals, and non-Latin script support as the primary remaining failure modes, motivating the future work directions described in Section 12.

The system is released under the MIT licence. The annotation guidelines and evaluation dataset will be made available to facilitate reproducibility and community extension of the benchmark.

References

[1] Qwen Team, Alibaba Cloud. *Qwen2.5 Technical Report*. arXiv preprint arXiv:2412.15115, 2024.

[2] H. Touvron et al. *Llama 2: Open Foundation and Fine-Tuned Chat Models*. arXiv preprint arXiv:2307.09288, 2023.

[3] Ollama. *Get up and running with large language models locally*. https://ollama.com, 2024.

[4] N. Reimers and I. Gurevych. *Sentence-BERT: Sentence Embeddings using Siamese BERT-Networks*. In *Proceedings of EMNLP-IJCNLP*, pp. 3982–3992, 2019.

[5] deepset GmbH. *Haystack: Neural Question Answering and Semantic Search at Scale*. https://haystack.deepset.ai, 2020.

[6] Microsoft Corporation. *Playwright: Fast and reliable end-to-end testing for modern web apps*. https://playwright.dev, 2020.

[7] L. Richardson. *Beautiful Soup: A Python library for pulling data out of HTML and XML files*. https://www.crummy.com/software/BeautifulSoup, 2004–2024.

[8] A. Barbaresi. *Trafilatura: A web scraping library and command-line tool for text discovery and extraction*. In *Proceedings of ACL-IJCNLP 2021: System Demonstrations*, pp. 122–131, 2021.

[9] N. Kushmerick. *Wrapper induction for information extraction*. PhD thesis, University of Washington, 1997.

[10] V. Crescenzi, G. Mecca, and P. Merialdo. *RoadRunner: Towards automatic data extraction from large web sites*. In *Proceedings of VLDB*, pp. 109–118, 2001.

[11] Y. Zhai and B. Liu. *Web data extraction based on partial tree alignment*. In *Proceedings of the 14th International Conference on WWW*, pp. 76–85. ACM, 2005.

[12] A. Finn, N. Kushmerick, and B. Smyth. *Fact or fiction: Content classification for digital libraries*. In *DELOS Workshop on Personalisation and Recommender Systems in Digital Libraries*, 2001.

[13] C. Lockard, P. Shiralkar, X. Dong, and H. Hajishirzi. *ZeroShotCeres: Zero-shot relation extraction from semi-structured webpages*. arXiv preprint arXiv:2005.07105, 2020.

[14] J. Wei et al. *Chain-of-thought prompting elicits reasoning in large language models*. In *Advances in Neural Information Processing Systems (NeurIPS)*, vol. 35, 2022.

[15] J. Li et al. *MarkupLM: Pre-training of text and markup language for visually-rich document understanding*. In *Proceedings of ACL*, pp. 4037–4051, 2022.

[16] Z. Zhou et al. *Document-level relation extraction via parallel multistage attention network*. In *Proceedings of EMNLP*, pp. 6890–6899, 2021.

[17] Q. McNemar. *Note on the sampling error of the difference between correlated proportions or percentages*. *Psychometrika*, 12(2):153–157, 1947.